

# Root Finding in One and Several Dimensions

Ehsan Al Twal

Student Number: 89614622

PHYS 410: Computational Physics

September 25, 2026

## 1 Introduction

Bisection is guaranteed to converge but only linearly; Newton's method converges quadratically but may diverge if started poorly. In Problem 1 I combine the two into a hybrid scheme and use it to find all ten roots on  $[0, 2]$  of

$$f(x) = 512x^{10} - 5120x^9 + 21760x^8 - 51200x^7 + 72800x^6 - 64064x^5 + 34320x^4 - 10560x^3 + 1650x^2 - 100x + 1. \quad (1)$$

In Problem 2 I generalize Newton's method to  $d$  dimensions and apply it to

$$x^2 + y^4 + z^6 = 2, \quad \cos(xyz^2) = x + y + z, \quad y^2 + z^3 = (x + y - z)^2, \quad (2)$$

near  $\mathbf{x}^{(0)} = (-1.00, 0.75, 1.50)$ .

## 2 Review of theory

**Bisection.** If  $f$  is continuous on  $[a_0, b_0]$  with  $f(a_0)f(b_0) < 0$ , the interval contains a root  $x^*$ . Each step forms the midpoint  $x_n = \frac{1}{2}(a_n + b_n)$  and keeps the half with the sign change, so

$$|x_n - x^*| \leq 2^{-(n+1)}(b_0 - a_0), \quad (3)$$

a linear rate of  $\log_{10} 2 \approx 0.30$  digits per step.

**Newton's method.** Linearizing  $f$  about the current estimate  $x_n$  and setting the result to zero gives the update

$$\delta_n = -\frac{f(x_n)}{f'(x_n)}, \quad x_{n+1} = x_n + \delta_n. \quad (4)$$

Each application of (4) is one step; the method repeats it until  $\delta_n$  is negligible. Near a simple root the error obeys  $e_{n+1} \approx [f''(x^*)/2f'(x^*)] e_n^2$ , i.e. the number of correct digits roughly doubles per step. In  $d$  dimensions the step becomes a linear system,

$$J(\mathbf{x}_n) \boldsymbol{\delta}_n = -\mathbf{f}(\mathbf{x}_n), \quad \mathbf{x}_{n+1} = \mathbf{x}_n + \boldsymbol{\delta}_n, \quad J_{ij} = \frac{\partial f_i}{\partial x_j}, \quad (5)$$

and convergence is again quadratic provided  $J$  is nonsingular at the root. There is no  $d$ -dimensional analogue of a bracket, so nothing plays the role of bisection in Problem 2.

**Exact roots of the test polynomial.** The leading coefficient  $512 = 2^9$  matches that of the Chebyshev polynomial  $T_{10}$ , and expanding  $T_{10}(u) = 512u^{10} - 1280u^8 + 1120u^6 - 400u^4 + 50u^2 - 1$  with  $u = x - 1$  reproduces (1) exactly, so  $f(x) = T_{10}(x - 1)$ . Since  $T_{10}$  vanishes at  $\cos[(2k - 1)\pi/20]$ ,

$$x_k^* = 1 + \cos\left(\frac{(2k - 1)\pi}{20}\right), \quad k = 1, \dots, 10, \quad (6)$$

which gives me an independent check on the computed roots.

### 3 Numerical approach

My function `hybrid` bisects until  $b_n - a_n \leq \text{tol1} \cdot |x_n|$ , then uses the final midpoint as the starting point for Newton's method, repeating (4) until  $|\delta_n| \leq \text{tol2} \cdot |x_{n+1}|$ . My function `newtond` repeats (5), solved with MATLAB's backslash operator, until  $\|\delta_n\| \leq \text{tol} \cdot \|\mathbf{x}_{n+1}\|$ . Both tests are relative, with a floor on the denominator in `hybrid` to cover a root at the origin, and both routines have iteration caps.

I find the initial brackets by brute force:  $[0, 2]$  is sampled at  $N = 10001$  points and each adjacent pair with  $f(x_i)f(x_{i+1}) < 0$  is recorded. The spacing  $\Delta x = 2 \times 10^{-4}$  is far below the smallest root separation ( $\approx 9.7 \times 10^{-2}$ ), so no root is missed. The function, derivative, residuals and Jacobian are all hard-coded analytically.

To reproduce the results, run `roots_hybrid` (Problem 1: Table 1, Figures 1 and 2) and `test_newtond` (Problem 2) from the MATLAB prompt in the HW1 directory. These scripts call `hybrid.m` and `newtond.m` respectively.

## 4 Results

### 4.1 Problem 1

With  $\text{tol1} = 10^{-5}$  and  $\text{tol2} = 10^{-12}$  the search found exactly ten brackets and `hybrid` converged on each (Table 1). Every root agrees with (6) to a relative accuracy between  $10^{-15}$  and  $6.2 \times 10^{-12}$ ; roots 1–7 meet the required  $10^{-12}$ , while roots 8–10 fall short by a factor of a few.

| $k$ | Computed root     | Exact root, Eq. (6) | $ f(x_k) $              |
|-----|-------------------|---------------------|-------------------------|
| 1   | 0.012311659404862 | 0.012311659404862   | $1.110 \times 10^{-16}$ |
| 2   | 0.108993475811632 | 0.108993475811632   | $1.443 \times 10^{-15}$ |
| 3   | 0.292893218813453 | 0.292893218813453   | $1.421 \times 10^{-14}$ |
| 4   | 0.546009500260421 | 0.546009500260453   | $1.359 \times 10^{-13}$ |
| 5   | 0.843565534959918 | 0.843565534959769   | $1.117 \times 10^{-12}$ |
| 6   | 1.156434465039169 | 1.156434465040231   | $8.584 \times 10^{-12}$ |
| 7   | 1.453990499739204 | 1.453990499739547   | $4.599 \times 10^{-11}$ |
| 8   | 1.707106781179397 | 1.707106781186547   | $7.774 \times 10^{-12}$ |
| 9   | 1.891006524176727 | 1.891006524188368   | $3.657 \times 10^{-10}$ |
| 10  | 1.987688340592448 | 1.987688340595138   | $7.740 \times 10^{-10}$ |

Table 1: Roots of Eq. (1) on  $[0, 2]$  from the hybrid algorithm, with  $\text{tol1} = 10^{-5}$  and  $\text{tol2} = 10^{-12}$ , compared against the exact values (6) and shown with the residual at each computed root.

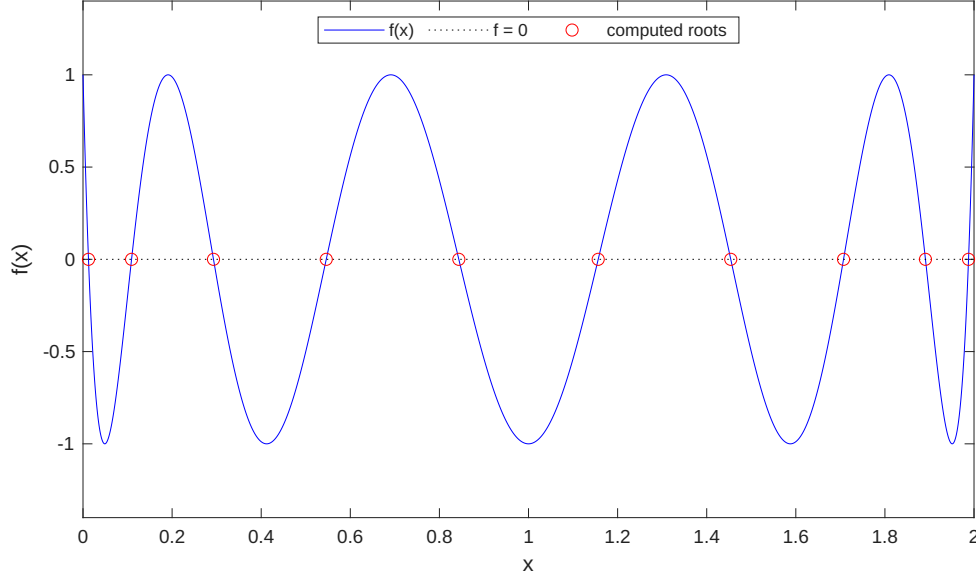


Figure 1: The test polynomial and its ten computed roots. Eq. (1) over  $[0, 2]$ , with the roots returned by `hybrid` marked as circles. As  $f(x) = T_{10}(x - 1)$ , the function oscillates between  $\pm 1$  with its zeros clustered towards the ends of the interval.

## 4.2 Problem 2

Starting from  $\mathbf{x}^{(0)} = (-1.00, 0.75, 1.50)$  with  $\text{tol} = 10^{-12}$ , `newtond` converged in nine iterations to

$$(x, y, z) = (-0.577705133337, 0.447447204972, 1.084412371725), \quad (7)$$

with residual  $\|\mathbf{f}\| = 2.5 \times 10^{-16}$ . The relative update at each step was

| $n$ | $\ \delta_n\ /\ \mathbf{x}_{n+1}\ $ | $n$ | $\ \delta_n\ /\ \mathbf{x}_{n+1}\ $ |
|-----|-------------------------------------|-----|-------------------------------------|
| 1   | $2.042 \times 10^{-1}$              | 6   | $5.686 \times 10^{-3}$              |
| 2   | $1.345 \times 10^{-1}$              | 7   | $4.265 \times 10^{-4}$              |
| 3   | $4.913 \times 10^{-2}$              | 8   | $7.024 \times 10^{-7}$              |
| 4   | $1.935 \times 10^{-1}$              | 9   | $4.010 \times 10^{-13}$             |
| 5   | $9.521 \times 10^{-2}$              |     |                                     |

From  $n = 6$  onward the exponent roughly doubles at each step, the signature of quadratic convergence.

## 5 Discussion and conclusions

Both of my routines performed as intended. In Problem 1 bisection reliably delivered an estimate inside Newton's basin of convergence, after which Newton reached `tol2` in a few steps, against the roughly twenty-three further halvings bisection would have needed.

The residuals in Table 1 grow by seven orders of magnitude from  $k = 1$  to  $k = 10$ , but comparison with the exact roots shows the roots themselves remain accurate. The growth comes from evaluating (1) near  $x = 2$ , where terms of size  $512 \times 2^{10} \approx 5 \times 10^5$  must cancel to something of order unity. This leaves a round-off floor of about  $10^{-16} \times 5 \times 10^5 \approx 10^{-10}$  on the computed  $f$ ,

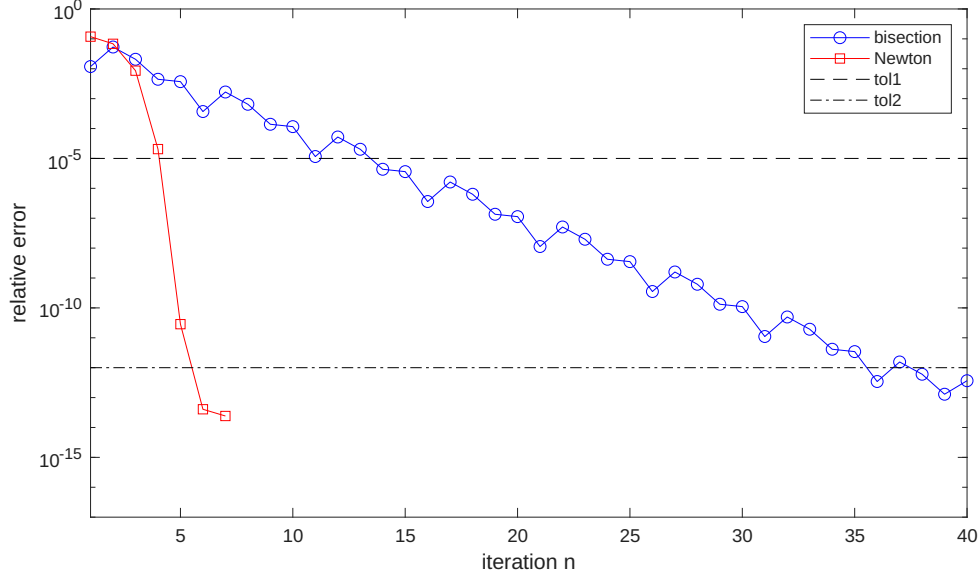


Figure 2: Linear versus quadratic convergence for the root near  $x = 0.8436$ . Relative error against iteration number for bisection and for Newton started from a comparable offset, with both tolerances marked. The bisection error descends raggedly, since (3) bounds it by a  $2^{-n}$  envelope without forcing a decrease at every step, but on average follows 0.30 digits per step. Newton reaches  $\sim 10^{-14}$  by  $n = 6$ , where it levels off at the accuracy of the reference value rather than of the method.

which is where the largest residuals sit. It also limits roots 8–10 to a relative accuracy of a few  $\times 10^{-12}$ , whatever the value of `tol2`. Writing  $f$  in the Chebyshev basis would avoid this.

In Problem 2 the relative update grows between  $n = 3$  and  $n = 4$  before recovering, so the given starting point is close enough for convergence but not for the quadratic rate to hold from the start. Nothing in several dimensions guards against this the way bisection does in Problem 1; damping the step until the residual decreases would.

The main problem I had was interpreting the large residuals near  $x = 2$  in Table 1. Until I noticed that  $f(x) = T_{10}(x - 1)$ , I could not tell whether they meant the roots there were inaccurate or were only round-off in evaluating  $f$ . Problem 2 has no exact solution to compare against, so I rely instead on the observed quadratic convergence and the machine-precision residual as evidence that `newton` found a genuine root.